

Stock Trading Chatbot Blueprint

Connected to Alpaca via Model Context Protocol (MCP)

Version 1.0 • February 6, 2026

Disclaimer: This document is for educational and engineering planning purposes only and is not investment advice. Automated trading can cause substantial losses. Use paper trading first and implement strict risk controls.

1. Hugging Face shortlist: bots and building blocks

Below are the most relevant projects/models for building an automated stock-trading chatbot. I grouped them into (A) end-to-end apps/bots and (B) model components you can combine into one decision engine.

1.1 End-to-end apps / bot templates

ParallelLLC/algorithmic_trading

Category: Full system template

What it gives you: FinRL-based algorithmic trading system with Alpaca integration, indicators, risk management, and UI options.

Hub link: https://huggingface.co/ParallelLLC/algorithmic_trading

Download: git clone https://huggingface.co/ParallelLLC/algorithmic_trading

Intel/stocktrader (Space)

Category: App template

What it gives you: Streamlit trading dashboard with sentiment + automated buy/sell loop and Alpaca execution (paper).

Hub link: <https://huggingface.co/spaces/Intel/stocktrader>

Download: git clone <https://huggingface.co/spaces/Intel/stocktrader>

rz2689/finrl-dapo-grpo-sentiment-risk

Category: RL agent checkpoint + code

What it gives you: FinRL Contest 2025 RL agent; integrates sentiment/risk ideas into reward shaping (research use).

Hub link: <https://huggingface.co/rz2689/finrl-dapo-grpo-sentiment-risk>

Download: git clone <https://huggingface.co/rz2689/finrl-dapo-grpo-sentiment-risk>

Adilbai/stock-trading-rl-agent

Category: RL agent checkpoint + preprocessing artifacts

What it gives you: PPO RL agent plus scaler/config files. Performance metrics are self-reported; validate via your own backtests.

Hub link: <https://huggingface.co/Adilbai/stock-trading-rl-agent>

Download: git clone <https://huggingface.co/Adilbai/stock-trading-rl-agent>

piccassol/NOLAND

Category: Agent architecture example (crypto)

What it gives you: Production-style agent stack (Solana) combining market feeds + social sentiment + LLM. Useful patterns even if you trade stocks.

Hub link: <https://huggingface.co/piccassol/NOLAND>

Download: git clone <https://huggingface.co/piccassol/NOLAND>

1.2 Model components (signals)

These are not complete bots by themselves. They're best used as modules that output a score or forecast that your main bot can combine.

ProsusAI/finbert

Category: Sentiment classifier

What it gives you: Financial sentiment classifier (positive/negative/neutral) for headlines, filings, and news.

Hub link: <https://huggingface.co/ProsusAI/finbert>

Download: `pip install transformers && python -c "from transformers import pipeline; print(pipeline('sentiment-analysis', model='ProsusAI/finbert')('Stocks rallied.'))"`

yyanghkust/finbert-tone

Category: Tone analysis

What it gives you: FinBERT fine-tuned on analyst report sentences (positive/negative/neutral).

Hub link: <https://huggingface.co/yyanghkust/finbert-tone>

Download: `pip install transformers && python -c "from transformers import pipeline; nlp=pipeline('sentiment-analysis', model='yyanghkust/finbert-tone'); print(nlp('growth is strong'))"`

FinGPT/fingpt-forecaster_dow30_llama2-7b_lora

Category: Forecasting adapter (LoRA)

What it gives you: LoRA adapter intended for market/finance forecasting prompts (requires a compatible Llama-2 base model and access).

Hub link: https://huggingface.co/FinGPT/fingpt-forecaster_dow30_llama2-7b_lora

Download: Use PEFT to load adapter on meta-llama/Llama-2-7b-chat-hf (see model card).

MasterDevYY/fingpt-forecaster_dow30_llama2-7b_lora-Q8_0-GGUF

Category: GGUF (llama.cpp)

What it gives you: GGUF conversion for llama.cpp workflows (LoRA GGUF).

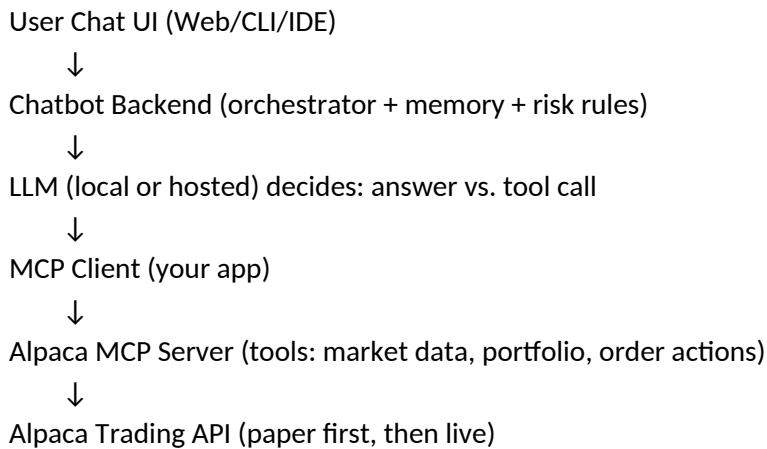
Hub link: https://huggingface.co/MasterDevYY/fingpt-forecaster_dow30_llama2-7b_lora-Q8_0-GGUF

Download: Download GGUF file(s) from the Files tab; use llama.cpp with `--lora`.

2. Target architecture

Goal: a chatbot interface that can research markets, produce trade plans, and (optionally) place orders via Alpaca's MCP Server—with strict safety guardrails and human confirmation before live orders.

High-level flow (recommended):



Key idea: only ONE component should submit orders. Everything else (signals, forecasts, RL agents) outputs structured recommendations that your meta-controller combines into a single trade proposal.

3. Alpaca MCP integration

Alpaca provides an official MCP server that exposes Alpaca Trading API capabilities (market data, order actions, portfolio details) to MCP-compatible clients. The server can be run locally or self-hosted remotely; remote hosting requires additional security controls (TLS + auth).

3.1 What the Alpaca MCP server can do

- Pull live and historical market data.
- Read portfolio state (positions, buying power, P/L).
- Place, modify, and cancel orders (including paper trading).
- Support equities, ETFs, crypto, and options depending on your Alpaca account/data subscriptions.

3.2 Client connection options

You can connect the MCP server to many clients (IDE assistants, CLIs, chat apps). Important constraint: ChatGPT currently supports only remote MCP servers (not local). For a custom chatbot you control, you can run the MCP client locally or in your own backend.

4. Build plan (phased)

Phase 0 - Paper trading only

1. Create/verify Alpaca paper trading account and generate paper API keys.
2. Install and run Alpaca MCP server locally.
3. Build a minimal chatbot that can: get quotes/bars, read account buying power, and draft an order payload.
4. Add an approval gate: the bot must present the order for manual confirmation before submission.

Phase 1 - Modular signal engine

5. Implement a unified Signal Schema (JSON) that every module outputs: direction, confidence, sizing hint, stop/take-profit, and rationale.
6. Plug in sentiment (FinBERT) for headlines/news you supply.
7. Plug in a forecasting module (FinGPT forecaster) for scenario text prompts (optional).
8. Optionally add an RL policy module (FinRL checkpoints) but treat it as one vote among others.
9. Create a Meta-Controller that combines signals using weighted rules and hard risk limits.

Phase 2 - Safety, logging, and test harness

10. Add trade limits: max position size, max daily loss, max open orders, symbol allowlist, hours restrictions.
11. Add robust audit logging (prompt, tool calls, order payload, broker response).
12. Add unit tests for every risk rule (orders that violate rules must be rejected).
13. Run a paper-trading burn-in (2-4 weeks) with small position sizing to validate behavior.

Phase 3 - Go-live checklist

14. Swap to live keys only after paper results and safety tests are acceptable.
15. Keep the human approval gate for all live orders (at least initially).
16. Monitor slippage, fills, and risk metrics daily; auto-disable on anomalies.
17. Document incident response: how to revoke keys and stop trading quickly.

5. Reference implementation outline (Python)

This is an opinionated, practical layout for a Python-first stack. You can adapt to Node/TS if you prefer.

5.1 Suggested repo structure

```
trading-chatbot/  
  app/  
    api.py          # FastAPI: /chat endpoint  
    settings.py     # env + config  
    llm.py          # local or hosted LLM wrapper  
    mcp_client.py   # MCP client session + tool invocations  
  tools/  
    alpaca_tools.py # tool wrappers (quote, bars, place_order, positions)  
  strategy/  
    signal_schema.py # Signal JSON contract  
    sentiment.py     # FinBERT module  
    forecaster.py    # FinGPT module  
    rl_policy.py     # optional FinRL module  
    meta_controller.py # combines signals + risk rules  
  risk/  
    limits.py       # caps and validations  
    approvals.py    # human confirmation gate  
  storage/  
    audit_log.py    # structured logs (JSONL)  
    state.py        # minimal memory / positions cache  
  infra/  
    docker/         # optional containers  
    deploy/         # optional cloud deploy scripts  
  README.md
```

5.2 Signal schema (example)

Keep it consistent so you can swap modules without changing the rest of the system.

```
{  
  "symbol": "AAPL",  
  "time": "2026-02-06T15:30:00Z",  
  "horizon": "intraday|swing|long",  
  "direction": "long|short|flat",  
  "confidence": 0.0-1.0,  
  "size_hint": "small|medium|large",  
  "stop_loss_pct": 0.0,  
  "take_profit_pct": 0.0,  
  "rationale": ["bullet", "points"],  
  "source": "sentiment|forecast|rl|technical"  
}
```

5.3 Order approval gate

A non-negotiable safety feature: before any order is sent, show a human-readable summary and require explicit confirmation.

- Minimum confirmation fields:
- Symbol, side, qty/notional, order type, limit/stop prices (if any)
- Account type (paper vs live) and buying power impact
- Risk summary (max loss, stop level, position as % of equity)
- Reason tags (which modules agreed, which disagreed)

6. Security and reliability

Trading bots are prime targets for prompt injection and credential leakage. Treat your MCP server like a production service even in paper mode.

- Keys: store Alpaca keys in environment variables / secrets manager; never hardcode.
- Remote hosting: require HTTPS/TLS + token-based authentication; do not expose unauthenticated MCP endpoints publicly.
- Tool allowlist: expose only the Alpaca tools you need; disable any file/system tools in the same agent session to reduce exploit chains.
- Rate limiting: add per-user and global rate limits on order placement endpoints.
- Kill switch: one command to disable trading and revoke/rotate keys.

7. References (source links)

- Alpaca MCP server documentation - <https://docs.alpaca.markets/docs/alpaca-mcp-server>
- Alpaca MCP server homepage - <https://alpaca.markets/mcp-server>
- ParallelLLC algorithmic_trading (Hugging Face) - https://huggingface.co/ParallelLLC/algorithmic_trading
- Intel stocktrader Space (Hugging Face) - <https://huggingface.co/spaces/Intel/stocktrader>
- rz2689 finrl-dapo-grpo-sentiment-risk (Hugging Face) - <https://huggingface.co/rz2689/finrl-dapo-grpo-sentiment-risk>
- Adilbai stock-trading-rl-agent (Hugging Face) - <https://huggingface.co/Adilbai/stock-trading-rl-agent>
- piccassol NOLAND (Hugging Face) - <https://huggingface.co/piccassol/NOLAND>
- ProsusAI FinBERT (Hugging Face) - <https://huggingface.co/ProsusAI/finbert>
- yiyanghkust finbert-tone (Hugging Face) - <https://huggingface.co/yiyanghkust/finbert-tone>
- FinGPT forecaster adapter (Hugging Face) - https://huggingface.co/FinGPT/finopt-forecaster_dow30_llama2-7b_lora
- MCP Python SDK (GitHub) - <https://github.com/modelcontextprotocol/python-sdk>
- MCP client quickstart - <https://modelcontextprotocol.io/quickstart/client>

- OpenAI Help: MCP in ChatGPT (beta) - <https://help.openai.com/en/articles/12584461-developer-mode-apps-and-full-mcp-connectors-in-chatgpt-beta>